

HiWi Selection Challenge

Ensuring Fairness in FL report for Tasks on FairTrade Extension

Harit Sarangi

Matriculation Number: 10066024

Email: `harit.sarangi@stud.uni-hannover.de`

July 16, 2026

Task 1 – Reproduce & Analysis

The FairTrade FL pipeline was replicated and executed on the Adult dataset, designating gender as the sensitive attribute. The code was run using the default configuration of 3 clients with a random data split (R3C) over 50 communication rounds. Each client completed 15 local epochs per round, optimizing a Lagrangian-constrained loss function.

The empirical results at the end of the 50 communication rounds, compared to the benchmarks reported in Table 2 of the paper, are summarized in Table 1.

Table 1: Comparison of Reproduced Metrics against Paper Benchmarks (Adult R3C)

Metric	Initial (Round 1)	Reproduced (Round 50)	Paper (Table 2)
Balanced Accuracy (BA)	50.00%	74.03%	79.45%
Statistical Parity Difference (DP)	0.00%	1.46%	9.13%

While the run achieved slightly lower classification accuracy than the paper (74.03% vs. 79.45%), the reproduced run obtained a significantly better fairness profile (1.46% demographic disparity compared to the paper’s 9.13%). This variation is a classic demonstration of operating at a different point on the Pareto frontier; the optimization path favored a tighter demographic parity constraint at a minor cost to overall predictive power.

Multi-Objective Optimization (MOO) in FairTrade: FairTrade models federated learning as a MOO problem because accuracy and fairness are in tension. Using BoTorch, it runs Multi-Objective Bayesian Optimization (MOBO) where a Gaussian Process surrogate models how the learning rate (η) and fairness multiplier (α) affect performance. Each round, the q-Expected Hypervolume Improvement (qEHVI) acquisition function selects the optimal (η, α) pair, which clients use to train under a constrained Lagrangian loss, eliminating manual tuning.

The Role of the Pareto Front: Since no single ideal solution exists, the algorithm targets the **Pareto Front**—the set of non-dominated configurations where improving accuracy inherently degrades fairness. This frontier guides BoTorch along the optimal compromise boundary while discarding sub-optimal hyperparameter configurations. This mapped trade-off enabled selecting a balanced deployment model, which maintains a competitive 74.03% balanced accuracy while driving demographic bias to a negligible 1.46%.

Task 2 – Extend the Fairness Evaluation

1. Intersectional Setup and Evaluation

To analyze intersectional bias, the evaluation framework was extended in `evaluate_intersection.py` to consider gender and race simultaneously using the Adult dataset. The selection rates ($P(\hat{y} = 1)$) and the Statistical Parity Difference (SPD) for individual attributes were computed, as well as for the four subgroups defined by their intersection: White Males, White Females, Non-White Males, and Non-White Females. For the intersectional comparisons, White Males were designated as the baseline privileged group.

The computed SPD values are presented in Table 2.

Table 2: Single-Attribute vs. Intersectional Statistical Parity Difference (SPD)

Subgroup / Comparison	Metric Type	Statistical Parity Difference (SPD)
Gender (Female vs. Male)	Single-Attribute	−0.1428
Race (Non-White vs. White)	Single-Attribute	−0.0887
White Female vs. White Male	Intersectional	−0.1383
Non-White Male vs. White Male	Intersectional	−0.0721
Non-White Female vs. White Male	Intersectional	−0.2049

2. Intersectional Analysis and Discussion

Does the model appear fairer in isolation? Yes, the model appears significantly fairer when evaluating sensitive attributes in isolation than when examining their intersection. Individually, race has an SPD of −0.0887 and gender has an SPD of −0.1428. However, at their intersection, the disparity for Non-White Females compared to White Males worsens dramatically to −0.2049, exposing a much larger equity gap that independent audits completely mask.

Why does this happen? This discrepancy occurs because single-attribute metrics average out performance across different sub-populations. For instance, when evaluating race alone, the relatively high selection rate of Non-White Males (22.48%) averages out and masks the much lower selection rate of Non-White Females (9.21%). This inflates the overall metrics for the “Non-White” category, hiding the compounding double-disadvantage faced by Non-White Females.

Furthermore, because the optimization penalties in Task 1 only target a single attribute (gender) independently, the training loop is completely blind to how features combine. The model is under no optimization pressure to correct for compounding disparities at the intersections, allowing multidimensional bias to go unpenalized.

3. Metric Source Citation

The Statistical Parity Difference (SPD) metrics were implemented manually following the standard mathematical definition of demographic parity evaluated in Verma and Rubin (2018) and formally constrained in the reductions framework of Agarwal et al. (2018).

Task 3 – Joint Optimization of Multiple Sensitive Attributes

1. Design Choice: Minimax (Max-SPD) Joint Optimization

To extend FairTrade to jointly optimize fairness across both **gender** and **race** without degrading system scalability, a **Minimax (Max-SPD) joint optimization** approach was designed and implemented.

Rather than restructuring the pipeline into a higher-dimensional multi-objective problem—which would expand the hypervolume search space to 3D, exponentially increase sample complexity, and stall BoTorch’s

GP convergence—the framework preserves the two-objective framework:

$$\begin{aligned} \text{Objective 1: } & \max_{\theta} \text{ Balanced Accuracy}(\theta) \\ \text{Objective 2: } & \min_{\theta} \mathcal{F}(\theta) = \max(|\text{SPD}_{\text{gender}}(\theta)|, |\text{SPD}_{\text{race}}(\theta)|) \end{aligned}$$

Under this formulation, the Multi-Objective Bayesian Optimization (MOBO) loop continues to efficiently trace a stable 2D Pareto frontier. On the client side, the Lagrangian optimizer dynamically targets the active demographic bottleneck by computing independent constraint penalties and backpropagating gradients exclusively on the worst-violating attribute path.

2. Structural Code Modifications

Operationalizing this joint framework required targeted code changes across three core files in the repository:

- **load_data_utilities.py**: Modified the dataset loaders and helper functions to extract, package, and return both **sex** and **race** arrays simultaneously during client partition initialization.
- **constraint.py**: Redesigned the forward pass of **DemographicParityLoss** to receive both attribute tensors, compute their corresponding constraint violations, and dynamically output the higher loss value as the minimax training bottleneck.
- **FairTrade.py**: Integrated both sensitive attributes into the client update loop and updated the global evaluator to compute independent test SPDs, output the maximum absolute violation as the primary metric, and return this scalar back to the BoTorch GP optimization loop.

3. Empirical Performance and Trade-off Analysis

The joint optimization run reached convergence at Round 50 with a final global **Balanced Accuracy** of 74.03%, while driving the worst-off metric (**Max-SPD**) down to a highly constrained 4.68% (represented by a gender SPD of 1.46% and race SPD of -4.68%). This highlights the engineering trade-offs detailed in Table 3.

Table 3: Engineering Trade-offs of the Minimax (Max-SPD) Fairness Formulation

Advantages	Disadvantages
Saves Sample Complexity: Restricting the Pareto front to two dimensions allows BoTorch’s surrogate model to converge efficiently, avoiding the curse of dimensionality associated with a 3D search space.	Feedback Blindness: The GP loop only receives feedback on the active bottleneck attribute. It is temporarily blind to the secondary attribute, which can lead to suboptimal search trajectories.
Worst-Case Guarantee: Actively minimizing the worst-performing attribute naturally forces the model to drag both violation profiles down concurrently.	Non-Smooth Optimization: The max operator is non-differentiable at crossover points. This can introduce gradient oscillations during client local SGD iterations.
Zero-Tuning Philosophy: Preserves FairTrade’s hands-free optimization philosophy by avoiding the need to manually tune objective weight hyperparameters (w_1, w_2).	No Priority Control: Offers no mechanism to selectively prioritize one sensitive dimension over another if required by specific legal, clinical, or social constraints.

Task 4 – Code Review & Bug Hunt

A meticulous review of the FairTrade codebase revealed concrete bugs and architectural discrepancies that completely halt execution and prevent replication out of the box.

1. Bug A: Variable Unpacking Mismatch in the Optimization Loop

- **File Path:** `FairTrade.py` (duplicated in `FairTrade-crypten.py`)
- **Location:** Main loop (around lines 268–270), during the communication round evaluation phase.

Involved Code Block:

```
if round == 0:
    objectives, bal_acc_, fairness_notion_ = evaluate(alpha)
else:
    objectives, bal_acc_, fairness_notion_ = evaluate(updated_alpha, updated_lr)
```

Impact on Results & Pipeline: The helper function `evaluate()` returns a single 2-element `torch.Tensor` of shape `[1, 2]`. Attempting to unpack this single-dimensional tensor into three separate variables (`objectives`, `bal_acc_`, and `fairness_notion_`) triggers a fatal python `ValueError` on the very first round. This syntax crash completely halts the training pipeline out of the box, making baseline replication impossible unless the unpacking structure is corrected.

Proposed Fix: Align the main communication loop with the actual mathematical payload returned by the evaluation function. Removing the redundant and unused unpacked variables resolves the error:

```
if round == 0:
    objectives = evaluate(alpha)
else:
    objectives = evaluate(updated_alpha, updated_lr)
```

2. Bug B: Undefined Class Reference in Causal (ATE) Loss Module

- **File Path:** `constraint.py`
- **Location:** Inside the `AverageTreatmentEffectLoss` constructor and forward pass definitions (lines 101 and 131).

Involved Code Block:

```
# Inside class AverageTreatmentEffectLoss(ConstraintLoss):
super(EqualOpportunityLoss, self).__init__(n_class=self.n_class, alpha=alpha,
                                           p_norm=p_norm)
```

Impact on Results & Methodological Claims: The referenced class `EqualOpportunityLoss` is entirely undefined in the repository. Running the pipeline with causal Average Treatment Effect constraints (`--fairness_notion 'ate'`) triggers an immediate `NameError`. This bug reveals that the causal fairness branch of the repository was never structurally tested before release. It suggests that boilerplate code was copied from an unrelated Equal Opportunity project, completely breaking the framework’s claimed causal optimization capabilities.

Proposed Fix: Re-align the `super()` inheritance pointers with the actual class namespace to restore correct polymorphic execution:

```
# In constructor:
super(AverageTreatmentEffectLoss, self).__init__(n_class=self.n_class,
                                                  alpha=alpha, p_norm=p_norm)

# In forward:
return super(AverageTreatmentEffectLoss, self).forward(X, out, sensitive, y=y)
```

References

- [1] Badar, M.; Sikdar, S.; Nejd, W.; and Fisichella, M. 2024. *FairTrade: Achieving Pareto-Optimal Trade-Offs between Balanced Accuracy and Fairness in Federated Learning*. In Proceedings of the AAAI Conference on Artificial Intelligence, 38(10), 10834–10842. <https://doi.org/10.1609/aaai.v38i10.28971>.
- [2] Agarwal, A.; Beygelzimer, A.; Dudík, M.; Langford, J.; and Wallach, H. 2018. *A reductions approach to fair classification*. In International Conference on Machine Learning, 60–69. PMLR.
- [3] Verma, S.; and Rubin, J. 2018. *Fairness definitions explained*. In 2018 IEEE/ACM International Workshop on Software Fairness (FairWare), 1–7. IEEE.
- [4] Google Gemini. 2026. *Assisted in code analysis, debugging, and LaTeX compilation*.